
Shatranj — Python

Rapport final de projet de programmation

Dries Amina
Meklat Sarah
Benachour Souhila
El Ghali Ayman
Marchoud Souhail

8 avril 2026

Introduction

Dans le cadre du cours de *Projet de Programmation*, nous avons développé `shatranj-python`, une implémentation complète du jeu de Shatranj en Python 3.12. Le Shatranj est l’ancêtre direct des échecs modernes, né en Perse sassanide aux VI^e–VII^e siècles [?]; ses règles de déplacement diffèrent sensiblement des pièces actuelles, ce qui en fait un sujet de programmation à part entière.

Ce rapport présente les règles du jeu et leur contexte historique (section ??), les structures de données et algorithmes employés (section ??), l’architecture logicielle (section ??), puis détaille deux fonctionnalités clés : l’intelligence artificielle (section ??) et l’interface graphique GTK (section ??). La section ?? dresse un bilan critique du projet.

1 Explication détaillée du sujet

1.1 Contexte historique

Le *chaturanga* indien, attesté dès le VI^e siècle [?], est l’ancêtre commun de tous les jeux d’échecs modernes. Il donna naissance au Shatranj en Perse sassanide; ce jeu est décrit en détail dans le *Kitāb al-Shatrānj* d’al-Adlī [?], traité du IX^e siècle qui codifie les règles, les problèmes (*mansūbāt*) et les ouvertures. Le Shatranj se diffusa en Europe médiévale et évolua vers les échecs modernes entre le XV^e et le XVI^e siècle [?, ?], notamment par l’acquisition par le Ferz de la mobilité illimitée de la Dame moderne [?].

La différence la plus frappante concerne précisément ce *Ferz* : il ne se déplace que d’une case en diagonale, là où la Dame moderne est la pièce la plus puissante [?]. L’Alfil, ancêtre du Fou, ne peut atteindre que huit cases fixes toutes de même couleur [?], contre une diagonale entière pour le Fou actuel.

1.2 Implémentations existantes

Fairy-Stockfish [?] est un moteur d’échecs variants basé sur Stockfish supportant le Shatranj via un système de pièces configurables; il utilise des bitboards 64 bits et l’algorithme NNUE. La librairie **python-chess** [?] permet de représenter des variantes d’échecs en Python mais ne supporte pas nativement le Shatranj. Notre implémentation se distingue par une interface graphique GTK 4 avec horloge intégrée, deux thèmes de pièces SVG, un CLI complet avec historique readline, et un moteur de règles spécifique au Shatranj (Bare King, promotion uniquement en Ferz), sans dépendance externe pour le moteur de jeu.

1.3 Le plateau et la notation

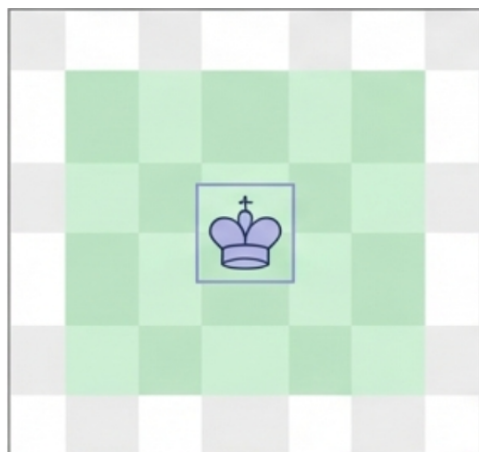
Le Shatranj se joue sur un échiquier 8×8 . Les colonnes sont désignées par a–h et les rangées par 1–8. En interne, les cases sont numérotées de 0 (a1) à 63 (h8) :

$$\text{numéro} = \text{rangée} \times 8 + \text{colonne}$$

Exemple. La case e4 : rangée 3, colonne 4, numéro $3 \times 8 + 4 = 28$.

1.4 Les pièces et leurs déplacements

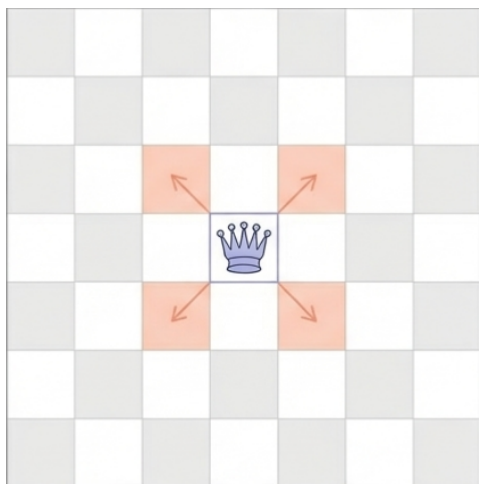
Le jeu comporte six types de pièces. Les blanches sont notées en majuscules, les noires en minuscules.



Shah (K/k) — Le Roi

Se déplace d'une case dans toutes les directions (horizontal, vertical, diagonal) [?]. C'est la pièce à protéger absolument : si le Shah est mis en échec et mat, la partie est perdue.

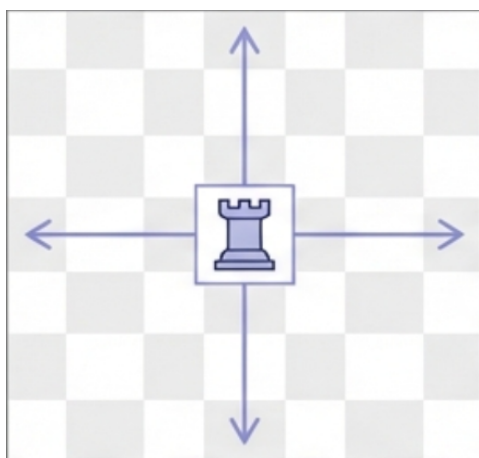
FIGURE 1 – Shah



Ferz (F/f) — Le Conseiller

Se déplace d'une *seule* case en diagonale [?]. C'est l'une des pièces les plus faibles du jeu.

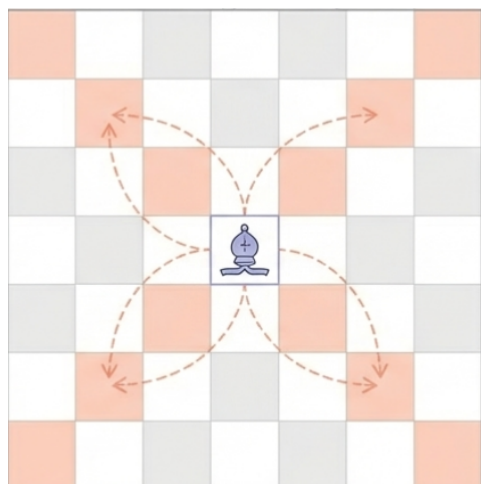
FIGURE 2 – Ferz



Rook (R/r) — La Tour

Se déplace en ligne droite (horizontale ou verticale) sur toute la longueur du plateau [?]. Bloquée par les pièces sur son chemin.

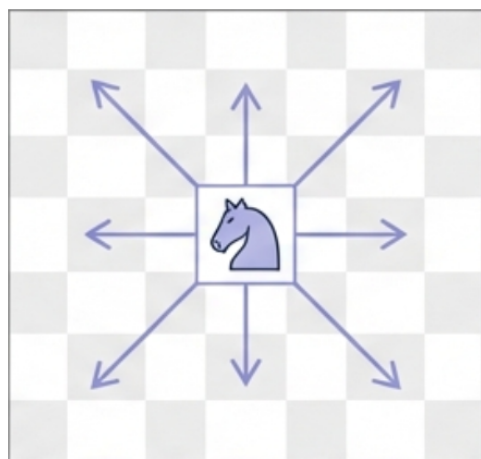
FIGURE 3 – Tour



Alfil (A/a) — L'Éléphant

Saute exactement deux cases en diagonale en franchissant la case intermédiaire [?]. Ne change jamais de couleur de case.

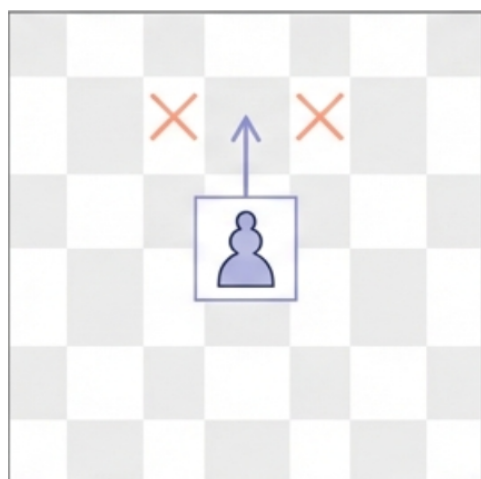
FIGURE 4 – Alfil



Knight (N/n) — Le Cavalier

Se déplace en « L » : deux cases dans une direction puis une case perpendiculaire. Peut sauter par-dessus les autres pièces.

FIGURE 5 – Cavalier



Pawn (P/p) — Le Pion

Avance d'une seule case vers l'avant. Capture en diagonale. Pas de double pas initial, ni de prise en passant. La promotion se fait uniquement en Ferz [?].

FIGURE 6 – Pion

1.5 Conditions de fin de partie

Le Shatranj connaît quatre conditions de fin de partie, dont deux (**Pat** et **Bare King**) diffèrent fondamentalement des échecs modernes.

Échec et Mat (Victoire)



FIGURE 7 – Échec et mat

Échec et mat

Le Shah adverse est en échec et ne peut s'échapper : aucun coup légal ne le sort de la menace. Le joueur qui donne mat gagne immédiatement.

Dans le code : `is_checkmate(board, color)` vérifie simultanément `is_in_check` et l'absence de coups légaux.

Le Pat (Match Nul)



FIGURE 8 – Pat (le joueur patté **perd**)

Pat — Règle propre au Shatranj

Contrairement aux échecs modernes où le pat est nul, au Shatranj le joueur n'ayant aucun coup légal **perd** la partie [?].

Dans le code : `is_stalemate(board, color)` retourne `True` si le joueur n'est pas en échec mais n'a aucun coup légal ; `_check_game_over()` attribue la victoire à l'adversaire.

Bare King (Victoire Immédiate)

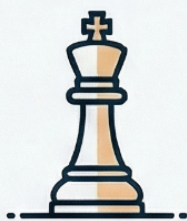


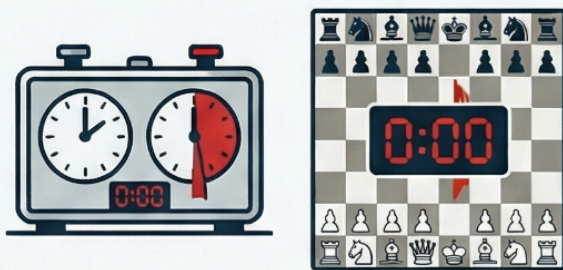
FIGURE 9 – Bare King (Shah nu)

Bare King (*Shah nu*)

Si toutes les pièces d'un joueur sont capturées (seul son Shah reste), il perd immédiatement, même si ce Shah n'est pas en échec [?].

Dans le code : `is_bare_king(board, color)` compte le nombre de bits actifs dans tous les bitboards du joueur hors Shah ; si ce compte vaut 0, la condition est déclenchée.

Timeout (Défaite au Temps)



Timeout — Mode Blitz

En mode blitz, chaque joueur dispose d'un capital de temps fixe. Lorsque le chronomètre d'un joueur atteint zéro, il perd immédiatement quelle que soit la position.

Implémenté dans `_finish_active_turn(moving_color)` si `remaining ≤ 0`, la victoire est accordée à l'adversaire.

FIGURE 10 – Timeout (mode Blitz)

1.6 Analyse de la complexité du jeu

Le facteur de branchement moyen est $b \approx 30$ (inférieur aux ≈ 35 des échecs modernes car le Ferz et l'Alfil sont très limités). L'élagage α - β ramène b^d à $b^{d/2}$ [?, ?] :

Profondeur d	Sans élagage (30^d)	Avec élagage ($30^{d/2}$)	Gain
3	27 000	164	$\times 165$
4	810 000	900	$\times 900$
5	24 300 000	5 477	$\times 4\,437$

2 Algorithmes et structures de données

2.1 Vue d'ensemble

Le projet mobilise les structures et algorithmes suivants :

- **Bitboards** : entiers 64 bits représentant l'état du plateau [?].
- **AlphaBeta** : Minimax optimisé, complexité $O(b^{d/2})$ [?].
- **Iterative Deepening** : approfondissement itératif avec limite de temps.
- **Zobrist hashing** : hachage incrémental en $O(1)$ [?].
- **Table de transposition** : mémorisation des positions.
- **MCTS** : recherche par simulations aléatoires [?].

2.2 Bitboards

2.2.1 Principe

Un *bitboard* est un entier non signé de 64 bits dans lequel chaque bit représente une case du plateau [?]. Le bit i vaut 1 si une pièce donnée occupe la case i . La classe `Board` maintient 12 bitboards (6 types de pièces $\times$ 2 couleurs).

2.2.2 Numérotation des cases

	a	b	c	d	e	f	g	h
8	56	57	58	59	60	61	62	63
7	48	49	50	51	52	53	54	55
⋮				...				
2	8	9	10	11	12	13	14	15
1	0	1	2	3	4	5	6	7

2.2.3 Implémentation Python : spécificités et astuces

En Python, les entiers sont de précision arbitraire : il n'existe pas de type `uint64` natif. Nous utilisons des entiers Python ordinaires et des masques explicites lorsque nécessaire. Le module `data/bitboards/bitboard.py` fournit les primitives avec validation systématique des indices via `check_square` :

```
def check_square(square: int) -> bool:
    if square < 0 or square >= NUM_SQUARES:    # NUM_SQUARES = 64
        raise InvalidSquareError(
            f"Square {square} must be in [0, {NUM_SQUARES - 1}]"
        )
    return True

def set_bit_at(bitboard: int, square: int) -> int:
    check_square(square)
    return bitboard | (1 << square)

def clear_bit_at(bitboard: int, square: int) -> int:
    check_square(square)
    return bitboard & ~(1 << square)

def get_bit_at(bitboard: int, square: int) -> int:
    check_square(square)
    return (bitboard >> square) & 1

def count_bits(bitboard: int) -> int:
    return bitboard.bit_count()    # méthode native Python 3.10+
```

Astuce : `bit_count()` natif Python 3.10+. Plutôt que d'utiliser `bin(x).count('1')`, nous utilisons la méthode `.bit_count()` introduite en Python 3.10, qui délègue au compilateur une instruction `popcount` optimisée sur les architectures modernes. Le gain est significatif dans l'évaluateur qui compte les pièces à chaque nœud.

Astuce : isolation du LSB en complément à deux.

```
def get_lsb(bitboard: int) -> int:
    if bitboard == 0:
        return -1
    return (bitboard & -bitboard).bit_length() - 1

def pop_lsb(bitboard: int) -> tuple[int, int]:
    idx = get_lsb(bitboard)
    if idx == -1:
        return -1, 0
    return idx, bitboard ^ (1 << idx)

def squares_from_bitboard(bitboard: int) -> list[int]:
    """Itère efficacement sur toutes les cases occupées."""
    squares = []
    bb = bitboard
    while bb:
        idx, bb = pop_lsb(bb)
        squares.append(idx)
    return squares
```

$x \& (-x)$ isole le bit le moins significatif en $O(1)$: en complément à deux, $-x = \neg x + 1$, donc tous les bits inférieurs au LSB sont mis à 0. `squares_from_bitboard` permet d'itérer sur toutes les cases occupées en $O(k)$ où k est le nombre de bits actifs.

2.2.4 Application et annulation d'un coup en $O(1)$

L'application d'un coup modifie exactement deux bits : on efface la case source et on active la case destination. En cas de capture, on efface en plus le bit adverse. Ces opérations sont toutes en $O(1)$:

```
def apply_move(self, move: Move) -> str | None:
    # Retirer la pièce de la case source
    self._boards[(move.piece_type, move.color)] = clear_bit_at(
        self._boards[(move.piece_type, move.color)], move.from_square)
    # La placer sur la case destination
    self._boards[(move.piece_type, move.color)] = set_bit_at(
        self._boards[(move.piece_type, move.color)], move.to_square)
    # Capturer l'éventuelle pièce adverse
    captured = None
    for piece in PIECES:
        if get_bit_at(self._boards[(piece, opponent)], move.to_square):
            self._boards[(piece, opponent)] = clear_bit_at(
                self._boards[(piece, opponent)], move.to_square)
            captured = piece
            break
    return captured    # retourné pour undo_move
```

undo_move est la symétrique : la pièce revient en from_square et la pièce capturée (si elle existait) est remise sur to_square.

2.2.5 Génération des coups légaux

```
FUNCTION generate_legal_moves(board, color):
    pseudo <- generate_pseudo_legal_moves(board, color)
    legal <- []
    FOR move IN pseudo:
        captured <- board.apply_move(move)          --  $O(1)$ 
        IF NOT is_in_check(board, color):            --  $O(P)$ 
            legal.append(move)
        board.undo_move(move, captured)              --  $O(1)$ 
    RETURN legal
```

Complexité. $O(P^2)$ avec $P \leq 32$ pièces — au plus 1 024 opérations élémentaires, négligeable par rapport aux nœuds explorés.

2.3 AlphaBeta (Minimax avec élagage)

L'algorithme Minimax [?] modélise le jeu comme un arbre alternant MAX (l'IA) et MIN (l'adversaire). L'élagage α - β [?] l'améliore : dès que $\beta \leq \alpha$, la branche est abandonnée.

```
FUNCTION alphabeta(board, depth, alpha, beta, is_max, color, opp):
    IF depth = 0 OR terminal(board):
        RETURN evaluate(board, color)
    IF is_max:
        FOR move IN legal_moves(board, color):
            apply(move)
            score <- alphabeta(board, depth-1, alpha, beta, FALSE, ...)
            undo(move)
            alpha <- max(alpha, score)
            IF beta <= alpha: BREAK          (* coupure beta *)
        RETURN alpha
    ELSE:
        FOR move IN legal_moves(board, opp):
            apply(move)
```

```

        score <- alphabeta(board, depth-1, alpha, beta, TRUE, ...)
        undo(move)
        beta <- min(beta, score)
        IF beta <= alpha: BREAK          (* coupure alpha *)
    RETURN beta

```

Déroulement. Supposons $\alpha = 5$ (MAX a un coup valant 5). Si MIN trouve 3 ($\beta = 3 \leq \alpha = 5$), cette branche est abandonnée : MAX ne la choisira jamais. À profondeur 4 : de $30^4 = 810\,000$ à ≈ 900 nœuds dans le meilleur cas [?].

2.4 Approfondissement itératif (Iterative Deepening)

Plutôt que de chercher directement à profondeur d , l'approfondissement itératif explore successivement les profondeurs 1, 2, ..., d . L'avantage principal est de toujours disposer d'un coup de repli si le temps s'écoule :

```

def best_move(self, board: Board, color: str) -> Move | None:
    legal_moves = self._engine.generate_legal_moves(board, color)
    best_move = legal_moves[0]          # coup de repli garanti
    start_time = time.time()
    for current_depth in range(1, self._depth + 1):
        if self._time_limit is not None:
            if time.time() - start_time >= self._time_limit:
                break                    # temps épuisé -> retourner le meilleur
        self._ab._depth = current_depth
        move = self._ab.best_move(board, color)
        if move is not None:
            best_move = move            # mise à jour avec la meilleure trouvée
    return best_move

```

Avantage de l'ordonnancement des coups. La table de transposition stocke les résultats de profondeur $d - 1$: lors de la recherche à profondeur d , les coups identifiés comme bons à la profondeur précédente sont explorés en premier, maximisant les coupures α - β .

2.5 Hachage de Zobrist et table de transposition

2.5.1 Principe général et motivation

Lors d'une recherche Alpha-Beta ou MCTS, le moteur explore des millions de nœuds. Or, la même position peut être atteinte par des séquences de coups différentes (*transpositions*) : sans mémorisation, ces positions seraient réévaluées inutilement. La **table de transposition** (TT) cache les résultats déjà calculés. Elle repose sur le **hachage de Zobrist** [?] : une méthode ingénieuse pour calculer un identifiant unique (clé de 64 bits) pour chaque position du plateau.

2.5.2 Construction de la table aléatoire

Au démarrage, on génère *une seule fois* un entier aléatoire de 64 bits pour chaque triplet (*type_pièce*, *couleur*, *case*) :

$$\text{table}[\text{pièce}][\text{couleur}][\text{case}] \in \{0, \dots, 2^{64} - 1\}$$

Le nombre total d'entrées est $6 \times 2 \times 64 = 768$, auquel s'ajoute un nombre aléatoire supplémentaire *black_to_move_key* indiquant quel joueur a le trait. On obtient ainsi 769 nombres aléatoires distincts. Le générateur est initialisé avec la graine 42 pour garantir la **reproductibilité** entre exécutions.

```

# Initialisation (une seule fois à l'import du module)
rng = Random(42)
for piece in ALL_PIECES:          # 6 types
    for color in ALL_COLORS:      # 2 couleurs

```



```

for square in range(64):    # 64 cases
    table[(piece, color, square)] = rng.getrandbits(64)
black_to_move_key = rng.getrandbits(64)

```

2.5.3 Calcul de la clé initiale et propriétés algébriques

La clé d'une position est le **XOR** de tous les nombres aléatoires correspondant aux pièces effectivement présentes sur le plateau :

$$Z(\text{position}) = \bigoplus_{(p,c,s) \in \text{plateau}} \text{table}[p][c][s] \oplus \begin{cases} \text{black_to_move_key} & \text{si c'est au tour des noirs} \\ 0 & \text{sinon} \end{cases}$$

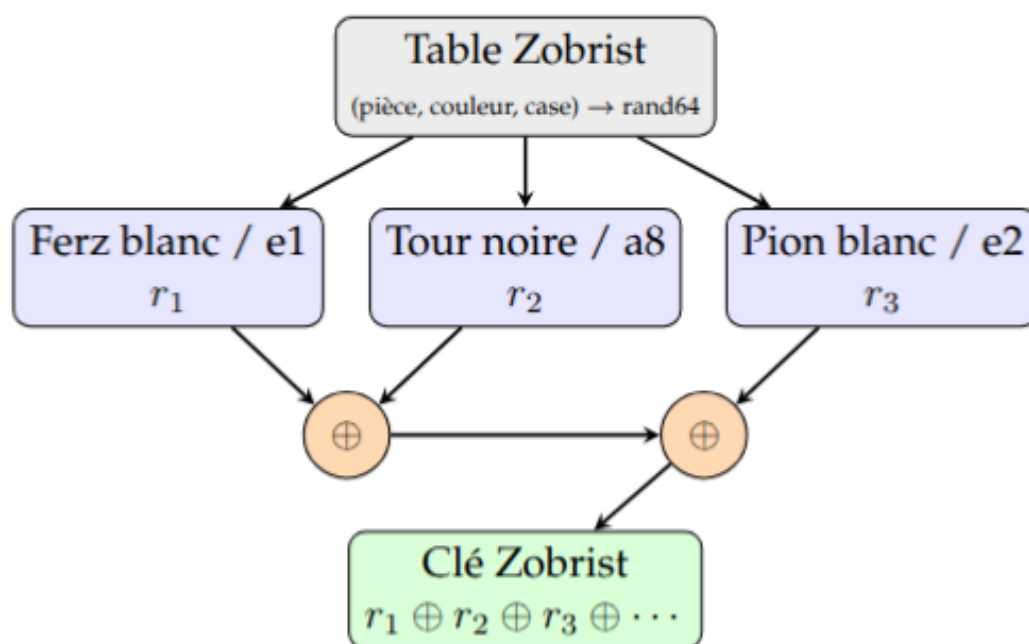


FIGURE 11 – Construction de la clé Zobrist par XOR des nombres aléatoires

Pourquoi XOR ? L'opération XOR possède trois propriétés cruciales pour ce contexte :

1. **Commutativité et associativité** : l'ordre d'application des pièces n'a pas d'importance ; toute position produit la même clé quelle que soit l'ordre de parcours du plateau.
2. **Auto-inverse** : $x \oplus x = 0$, donc XORer deux fois le même nombre l'annule exactement. Retirer une pièce ou l'ajouter utilisent la même opération.
3. **Complexité constante** : une opération XOR sur 64 bits prend $O(1)$ quelle que soit la taille des valeurs.

2.5.4 Mise à jour incrémentale après un coup — $O(1)$

C'est ici que réside l'avantage décisif de Zobrist. Lorsqu'une pièce se déplace de `from_square` vers `to_square`, la clé est mise à jour avec **au maximum 3 XOR** :

```

def update_key_for_move(key: int, move: Move,
                        captured_piece=None) -> int:
    # 1. Retirer la pièce de sa case source (XOR = annulation)
    key ^= table[(move.piece_type, move.color, move.from_square)]
    # 2. La placer sur sa case destination (XOR = ajout)

```

```

key ^= table[(move.piece_type, move.color, move.to_square)]
# 3. Supprimer la pièce capturée (le cas échéant)
if captured_piece is not None:
    opponent = BLACK if move.color == WHITE else WHITE
    key ^= table[(captured_piece, opponent, move.to_square)]
# 4. Changer de joueur
key ^= black_to_move_key
return key

```

Comparaison avec un hachage naïf. Un hachage naïf (ex. `hash(str(board))`) nécessiterait de parcourir les 64 cases à chaque nœud : $O(64) = O(1)$ mais avec une constante 64 fois plus grande. À profondeur 4, l'algorithme explore ≈ 900 nœuds ; l'économie est d'environ $900 \times 62 = 55\,800$ opérations par appel.

Undo en $O(1)$. La propriété auto-inverse du XOR garantit que `undo_move` restaure la clé sans recalcul : il suffit de rejouer les mêmes XOR dans le même ordre.

2.5.5 Probabilité de collision

Deux positions différentes pourraient théoriquement produire la même clé de 64 bits (*fausse positive*). La probabilité est :

$$P(\text{collision}) \approx \frac{1}{2^{64}} \approx 5.4 \times 10^{-20}$$

Avec un million de positions en mémoire, la probabilité qu'une paire quelconque entre en collision reste inférieure à $\sim 10^{-8}$. En pratique, les collisions Zobrist n'ont jamais été observées et n'affectent pas la jouabilité.

2.5.6 Structure de la table de transposition

La TT associe chaque clé Zobrist à un triplet (**score**, **profondeur**, **flag**) :

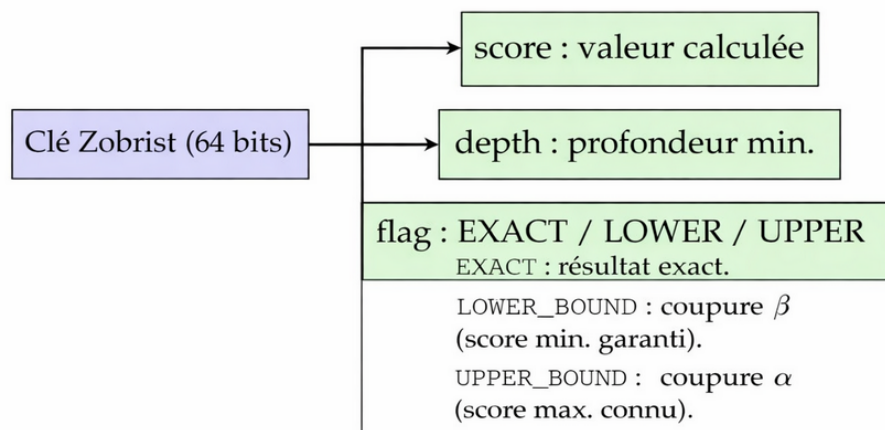


FIGURE 12 – Structure d'une entrée dans la table de transposition

2.5.7 Consultation et mise à jour dans AlphaBeta

```

# --- Consultation de la TT au début de chaque nœud ---
entry = self._table.get(key)
if entry and entry["depth"] >= depth:
    if entry["flag"] == EXACT:
        return entry["score"], True # résultat exact disponible
    if entry["flag"] == LOWER_BOUND and entry["score"] >= beta:

```

```

        return entry["score"], True          # coupure beta possible
    if entry["flag"] == UPPER_BOUND and entry["score"] <= alpha:
        return entry["score"], True          # coupure alpha possible

# --- Mise à jour après calcul ---
flag = EXACT
if score <= alpha:
    flag = UPPER_BOUND    # score maximum (alpha n'a pas bougé)
elif score >= beta:
    flag = LOWER_BOUND    # score minimum (coupure beta)
self._table.store(key, score, depth, flag)

```

2.5.8 Politique de remplacement et partage entre algorithmes

La TT est limitée à **10 000 entrées** avec la stratégie *always-replace* : si la table est pleine, la nouvelle entrée remplace simplement l'ancienne. Cette politique favorise les positions récemment vues, ce qui est cohérent avec l'approfondissement itératif.

La TT est **partagée** entre AlphaBeta, Iterative Deepening et MCTS :

- **AlphaBeta** → **MCTS** : les positions bien évaluées par AlphaBeta profitent directement aux rollouts du MCTS (consultation avant simulation).
- **Iterative Deepening** → **AlphaBeta** : les résultats de profondeur $d - 1$ guident l'ordonnancement des coups à profondeur d , maximisant les coupures.

Sur les positions répétées (ouvertures, structures de fin de partie), la TT réduit le nombre de nœuds évalués de **30 à 50 %**.

2.5.9 Synthèse des propriétés Zobrist

Propriété	Valeur / Comportement	Impact
Complexité calcul initial	$O(64)$ à la création du plateau	Fait une seule fois
Mise à jour après coup	$O(1)$ (2–3 XOR)	Critique en recherche
Undo	$O(1)$ (mêmes XOR)	Symétrique
Taille de la table	769 entiers 64 bits ≈ 6 Ko	Négligeable
Probabilité de collision	$\approx 5 \times 10^{-20}$	Ignorable
Reproductibilité	Graine fixe = 42	Débogage facilité
Partage entre algos	Oui (AlphaBeta + MCTS)	Synergies

3 Architecture logicielle

3.1 Principe N-tiers

L'application suit une architecture en couches strictement séparées [?] où chaque couche ne communique qu'avec la couche immédiatement inférieure. La figure ?? illustre les trois couches.

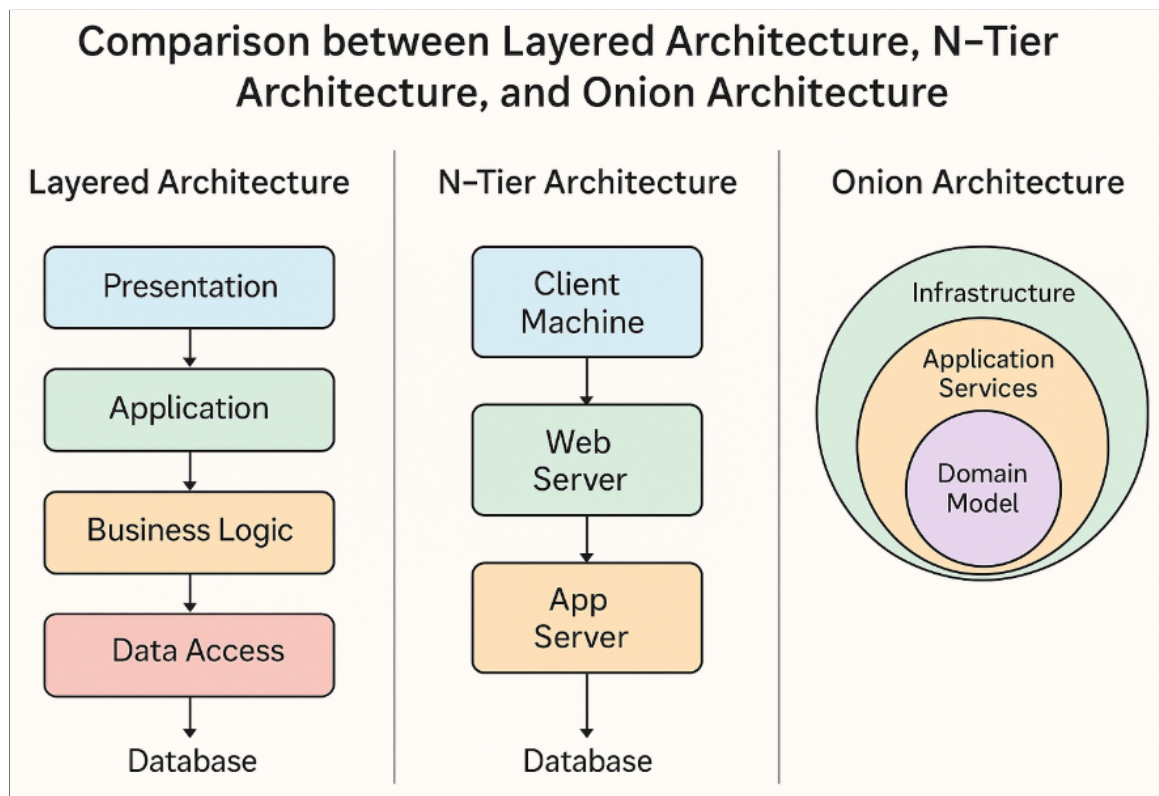


FIGURE 13 – Architecture en couches du projet Shatranj

La **couche Présentation** regroupe le CLI et le GUI GTK 4 : elle ne contient aucune règle de jeu. La **couche Métier** (Domain) est le cœur du programme : moteur de règles, algorithmes d'IA, gestion de l'état via les bitboards. La **couche Données** gère la persistance (format texte ASCII) et les primitives bit-à-bit.

3.2 Flux d'interactions

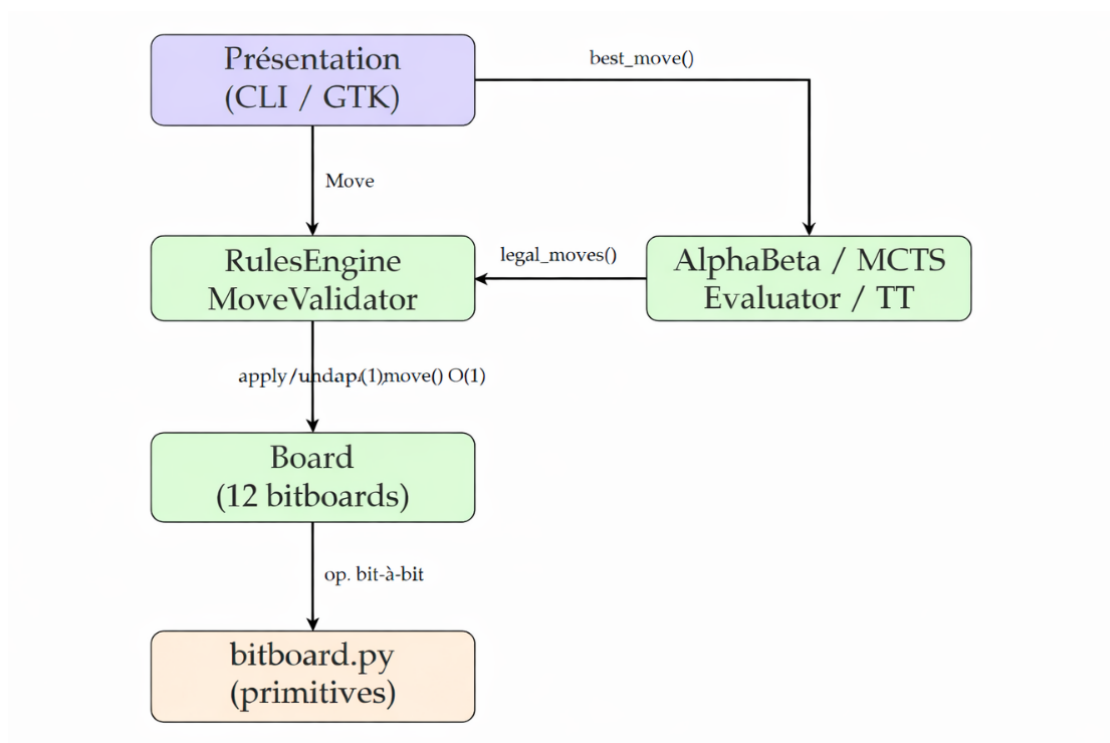


FIGURE 14 – Flux d'interaction entre les couches

Lorsqu'un utilisateur joue un coup (CLI ou GUI), la couche **Présentation** construit un objet `Move` et le transmet au **RulesEngine**. Celui-ci vérifie la légalité via `MoveValidator`, puis appelle `Board.apply_move()` qui modifie les bitboards en $O(1)$. Pour l'IA, `AIPlayer.choose_move()` sélectionne l'algorithme configuré (AlphaBeta, Iterative Deepening ou MCTS), consulte la `TranspositionTable`, et renvoie le meilleur coup. Dans le GUI, l'IA s'exécute dans un thread dédié afin de ne pas bloquer l'interface.

3.3 Principales interfaces entre couches — Schéma d'API

Plutôt qu'une liste textuelle, le schéma ci-dessous représente les quatre interfaces contractuelles entre les couches, avec la direction du flux de données et les types échangés.

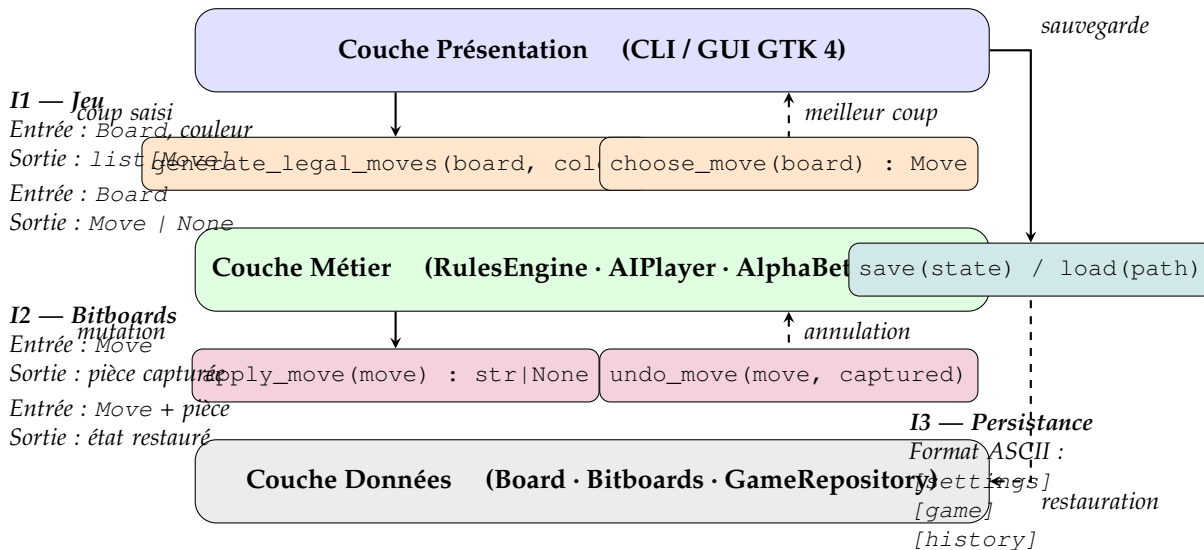


FIGURE 15 – Schéma des interfaces contractuelles entre les trois couches

Lecture du schéma. Les flèches pleines ($\rightarrow$) indiquent le sens de l'appel ; les flèches pointillées ($\leftarrow$) indiquent le retour de valeur. Chaque couche n'est autorisée à importer que les symboles de la couche inférieure, ce qui garantit une séparation des responsabilités stricte et facilite les tests unitaires.

- **I1 — Interface Jeu** : `RulesEngine.generate_legal_moves(board, color)` est le point d'entrée principal de la Présentation ; `AIPlayer.choose_move(board)` retourne le coup calculé par l'algorithme configuré (AlphaBeta, ID, MCTS).
- **I2 — Interface Bitboards** : `Board.apply_move(move)` modifie les bitboards en $O(1)$ et retourne la pièce capturée ; `undo_move(move, captured)` est la symétrie exacte.
- **I3 — Interface Persistance** : `GameRepository.save(state) / load(path)` sérialisent l'état complet (plateau, horloge, historique) au format ASCII structuré. Le GUI réutilise directement la logique de parsing du CLI (`CLI._do_load`).

4 Fonctionnalité approfondie 1 : Intelligence Artificielle

4.1 Vue d'ensemble

Le module `domain/ai/` propose quatre algorithmes : AlphaBeta (profondeur 4), Minimax basique, IterativeDeepening (profondeur 4 avec limite de temps) et MCTS (500 simulations par coup). Tous partagent le même Evaluator et la même TranspositionTable.

4.2 Fonction d'évaluation

L'Evaluator propose trois niveaux :

1. **Material** : somme algébrique des valeurs (Pion=1, Alfil=Ferz=2, Cavalier=6, Tour=9) [?].
2. **Positional** : material + bonus positionnel par table de 64 valeurs pour chaque type de pièce ; tables miroirs pour les noirs.
3. **Advanced** : positional + mobilité (nombre de coups légaux), contrôle du centre (+2 pts pour d4/e4/d5/e5), sécurité du Shah (malus si proche du centre).

4.3 Monte Carlo Tree Search (MCTS)

4.3.1 Principe et schéma

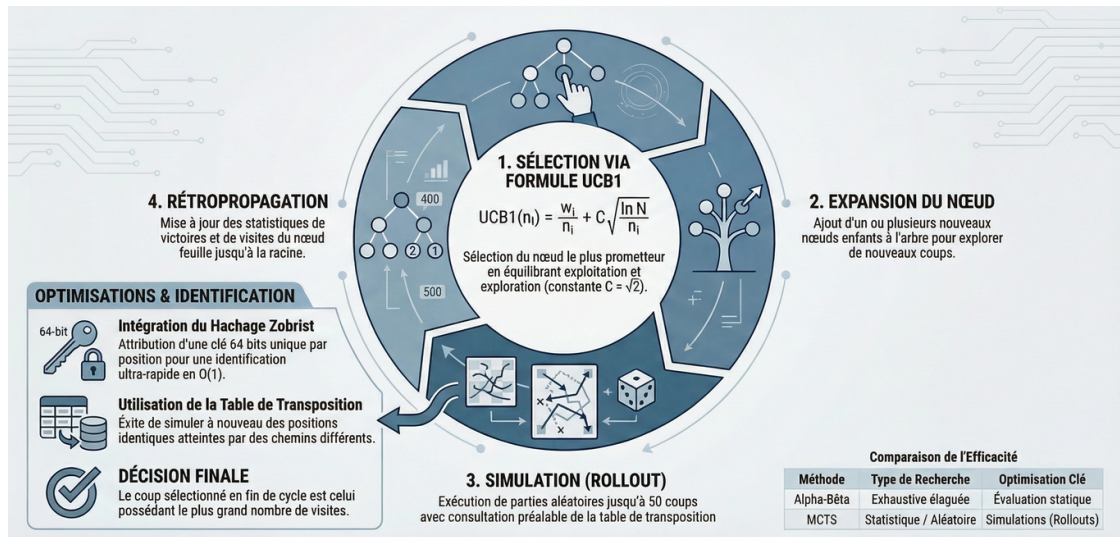


FIGURE 16 – Les quatre phases du MCTS : sélection, expansion, simulation, rétropropagation [?]

Le MCTS [?] évalue les positions par simulations aléatoires. À la différence d'AlphaBeta qui nécessite une fonction d'évaluation statique, le MCTS tire sa force de l'accumulation de milliers de parties simulées.

4.3.2 Structure MCTSNode

Chaque nœud de l'arbre conserve les statistiques nécessaires à UCB1 :

```
class MCTSNode:
    def __init__(self, move, parent, color):
        self.move = move
        self.parent = parent
        self.color = color
        self.children : list["MCTSNode"] = []
        self.wins : float = 0.0 # victoires simulées depuis ce nœud
        self.visits: int = 0 # nombre de passages
        self.untried: list[Move] = [] # coups non encore explorés

    def is_fully_expanded(self) -> bool:
        return len(self.untried) == 0
```

4.3.3 Formule UCB1 et sélection

$$UCB1(n_i) = \frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}, \quad C = \sqrt{2}$$

Le terme $\frac{w_i}{n_i}$ favorise l'exploitation des bons nœuds ; $C \sqrt{\frac{\ln N}{n_i}}$ favorise l'exploration des nœuds peu visités.

```
def best_child(self, exploration: float = 1.41) -> "MCTSNode":
    return max(
        self.children,
        key=lambda c: (c.wins / c.visits)
        + exploration * math.sqrt(math.log(self.visits) / c.visits),
    )
```

4.3.4 Rollout avec table de transposition

Notre MCTS intègre la table de transposition dans la phase de simulation : avant de lancer un rollout aléatoire, on consulte la TT pour éviter de rejouer des positions déjà connues :

```
def _rollout(self, board, current_color, ai_color, max_moves=50):
    color = current_color
    key = self._hasher.compute_key(board, color)
    # Consultation TT : évite les rollouts redondants
    tt_score, should_use = self._tt.get(key, 0, -inf, +inf)
    if should_use:
        return tt_score # position déjà connue -> résultat immédiat
    for _ in range(max_moves):
        legal_moves = self._engine.generate_legal_moves(board, color)
        if not legal_moves:
            opp = BLACK if color == WHITE else WHITE
            result = 1.0 if opp == ai_color else 0.0
            self._tt.store(key, result, 0, EXACT)
            return result
        if self._engine.is_bare_king(board, color):
            opp = BLACK if color == WHITE else WHITE
            result = 1.0 if opp == ai_color else 0.0
            self._tt.store(key, result, 0, EXACT)
            return result
        move = random.choice(legal_moves)
        board.apply_move(move)
        color = BLACK if color == WHITE else WHITE
    self._tt.store(key, 0.5, 0, EXACT)
    return 0.5
```

4.3.5 Rétropropagation et choix final

```
def _backpropagate(self, node, result: float) -> None:
    while node is not None:
        node.visits += 1
        node.wins += result
        result = 1.0 - result # inversion de perspective parent/enfant
        node = node.parent
```

Le coup final est le plus *visité* (non le meilleur ratio victoires/visites) : la fréquence de visite est un indicateur plus robuste [?]. Le MCTS utilise également une copie légère du plateau pour chaque simulation (`_copy_board` copie uniquement le dictionnaire de bitboards), ce qui évite les copies profondes coûteuses.

5 Fonctionnalité approfondie 2 : Interface graphique GTK 4

5.1 Vue d'ensemble

L'interface graphique est réalisée avec **PyGObject** [?], le binding Python de GTK 4. Elle reproduit toutes les fonctionnalités du CLI dans une fenêtre visuelle : nouvelle partie avec dialogue de configuration, horloge intégrée (Bullet/Blitz/Rapid/Custom), undo/redo, sauvegarde, hint, mode IA avec thread dédié.

5.2 Architecture du widget plateau

Le widget `BoardWidget` hérite de `Gtk.DrawingArea` et utilise **Cairo** pour le rendu et **librsvg** (`Rsvg.Handle`) pour charger les pièces au format SVG :

```
class BoardWidget(Gtk.DrawingArea):
    def __init__(self, engine: RulesEngine) -> None:
        super().__init__()
```

```

self._pieces = self._load_pieces()      # cache SVG en mémoire
self.set_draw_func(self._draw)          # callback de rendu Cairo
click = Gtk.GestureClick.new()
click.connect("pressed", self._on_click)
drag = Gtk.GestureDrag.new()
drag.connect("drag-begin", self._on_drag_begin)
drag.connect("drag-update", self._on_drag_update)
drag.connect("drag-end", self._on_drag_end)
self.add_controller(click)
self.add_controller(drag)

```

5.2.1 Dialogue de configuration

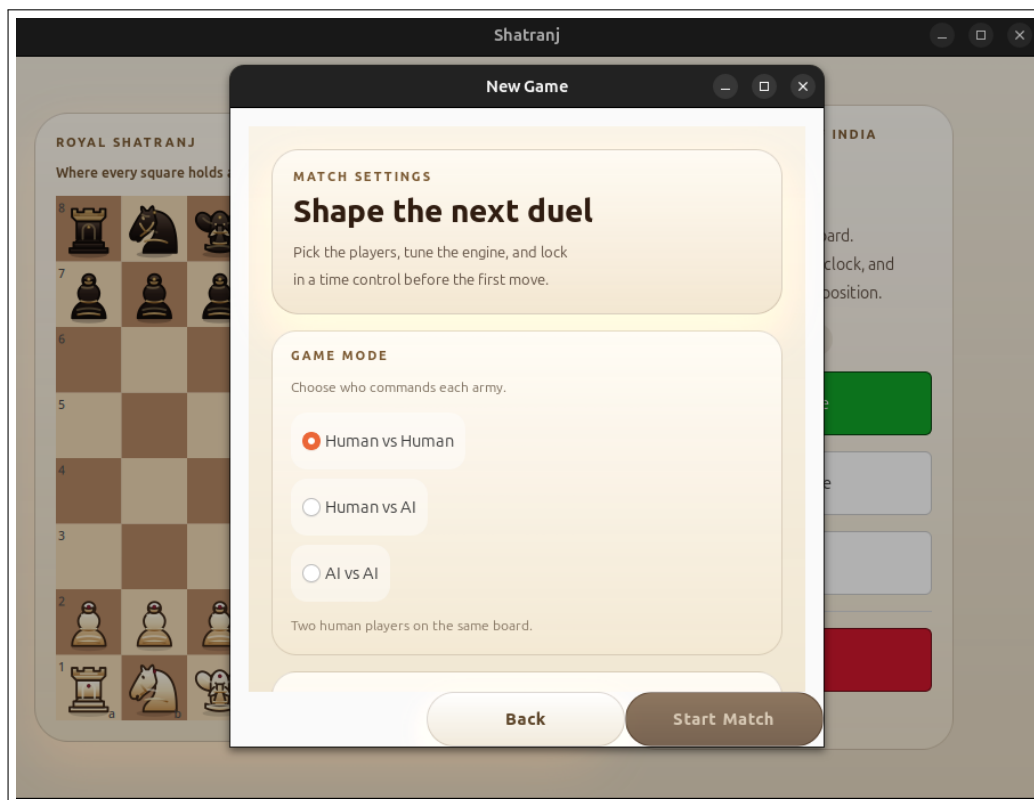


FIGURE 17 – Dialogue de configuration d’une nouvelle partie

5.2.2 Rendu Cairo + SVG vectoriel

Chaque pièce est chargée une seule fois via `Rsvg.Handle.new_from_file()` et mise en cache dans `self._pieces`. Le rendu à l’écran utilise `handle.render_cairo(cr)` après une transformation de coordonnées (translation + échelle). Le scaling est calculé dynamiquement depuis la taille du widget :

```

def _board_geometry(width, height):
    """Retourne la taille d'une case et les offsets pour centrer le plateau."""
    board_size = min(width, height)
    square_size = board_size / BOARD_SIZE      # BOARD_SIZE = 8
    offset_x = (width - board_size) / 2
    offset_y = (height - board_size) / 2
    return square_size, offset_x, offset_y

```

Ce calcul garantit que le plateau reste carré et centré quelle que soit la taille de la fenêtre. Les pièces sont mises à l’échelle dynamiquement :

```

has_size, svg_w, svg_h = handle.get_intrinsic_size_in_pixels()
if not has_size or svg_w == 0 or svg_h == 0:
    svg_w, svg_h = 45.0, 45.0 # fallback si SVG sans taille
scale = sq / max(svg_w, svg_h) # sq = taille d'une case en pixels
cr.save()
cr.translate(x, y)
cr.scale(scale, scale)
handle.render_cairo(cr)
cr.restore()

```

La formule $scale = sq / \max(svg_w, svg_h)$ assure que quelle que soit la taille intrinsèque du SVG (45×45, 128×128, etc.), la pièce occupe *exactement* une case. Toutes les pièces ont ainsi des dimensions **identiques** à l'écran, ce qui garantit un rendu cohérent et visuellement propre.

Deux thèmes de pièces. Le widget sélectionne automatiquement le thème `pieces_royal/` s'il existe, et se replie sur `pieces/` sinon. Les SVG utilisent la notation standard : `wK.svg` (Shah blanc), `bA.svg` (Alfil noir), etc.

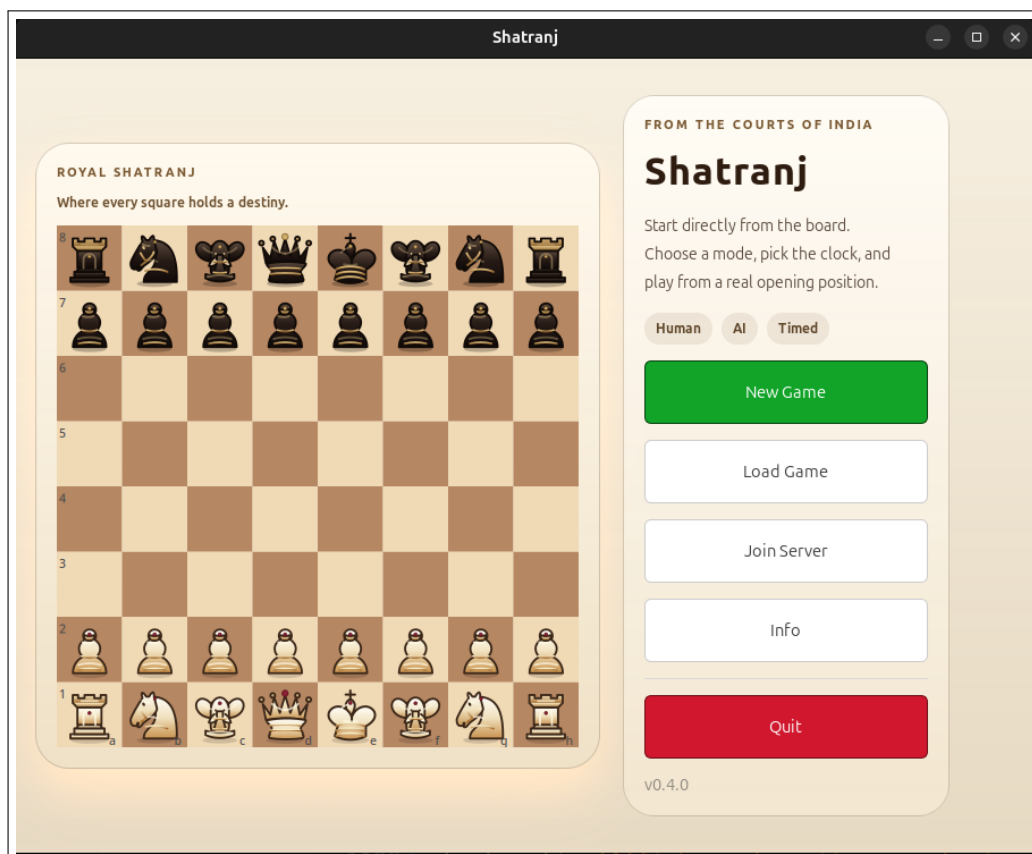


FIGURE 18 – Plateau avec pièces SVG (thème Royal)

5.3 Affichage des pièces capturées — Détail technique

5.3.1 Architecture des bandeaux de captures

Les pièces capturées sont affichées dans deux **strips horizontaux** (bandeaux) positionnés respectivement au-dessus et en dessous du plateau. Le strip du haut affiche les pièces blanches capturées par les noirs ; celui du bas affiche les pièces noires capturées par les blancs.

5.3.2 Implémentation — De l'historique aux icônes

La logique de calcul des captures est scindée en deux fonctions :

Méthode primaire : `_captured_pieces_from_history`. Cette méthode parcourt la liste des coups joués (`state.get_history()`) et collecte chaque pièce dont `move.captured_piece` est non nul. Elle est fiable car elle reflète exactement le déroulement de la partie, y compris après un undo/redo :

```
def _captured_pieces_from_history(history) -> dict[str, list[str]] | None:
    captured = {WHITE: [], BLACK: []}
    for move in history:
        if not move.captured_piece:
            continue
        # La pièce capturée appartient à l'adversaire du joueur qui a joué
        captured_color = BLACK if move.color == WHITE else WHITE
        captured[captured_color].append(move.captured_piece)
    return {
        color: _sort_captured_piece_types(captured[color])
        for color in (WHITE, BLACK)
    }
```

Méthode de repli : `_captured_pieces_from_board`. Pour les sauvegardes anciennes dont l'historique ne contient pas les captures, on déduit les pièces manquantes en comparant le nombre de pièces sur le plateau avec les effectifs de départ :

```
STARTING_PIECE_COUNTS = {FERZ: 1, ROOK: 2, ALFIL: 2, KNIGHT: 2, PAWN: 8}

def _captured_pieces_from_board(board) -> dict[str, list[str]]:
    remaining = {WHITE: {...}, BLACK: {...}} # compte les pièces actuelles
    captured = {WHITE: [], BLACK: []}
    for color in (WHITE, BLACK):
        for piece in CAPTURE_DISPLAY_ORDER: # ordre visuel stable
            missing = STARTING_PIECE_COUNTS[piece] - remaining[color][piece]
            captured[color].extend([piece] * max(0, missing))
    return captured
```

Affichage — `_make_captured_piece_widget`. Chaque pièce capturée est représentée par une icône de 20×20 pixels (taille fixe), construite depuis le même fichier SVG que le plateau :

```
def _make_captured_piece_widget(self, piece: str, color: str):
    path = get_piece_asset_path(piece, color) # même thème que le plateau
    if path is not None:
        picture = Gtk.Picture.new_for_filename(path)
        picture.set_size_request(20, 20) # dimensions fixes identiques
        picture.set_can_shrink(True)
        picture.set_keep_aspect_ratio(True) # ratio SVG préservé
        picture.set_tooltip_text(
            PIECE_LABELS.get(piece, piece.title())
        )
        return picture
    # Repli textuel si le SVG est introuvable
    label = Gtk.Label(label=piece[:1].upper())
    return label
```

Point clé — Cohérence des dimensions. La taille de 20×20 est choisie pour que les captures restent discrètes et ne rivalisent pas visuellement avec le plateau. La méthode `set_keep_aspect_ratio(True)` garantit que les SVG de formes variées (pion court, tour large) ne sont jamais déformés; la méthode `set_can_shrink(True)` permet à GTK de réduire l'image si l'espace devient insuffisant. Ces deux appels combinent le même principe que sur le plateau (`scale = sq / max(svg_w, svg_h)`) mais délèguent le calcul à GTK plutôt qu'à Cairo.

Rafraîchissement. `_update_captured_pieces()` est appelée systématiquement depuis `_refresh_game_vie` après chaque coup, undo ou chargement. Elle vide d’abord les strips (`_clear_box_children`) puis les remplit avec les nouvelles icônes :

```
def _update_captured_pieces(self) -> None:
    captured = _captured_pieces_for_display(self._state)
    # strip du haut = pièces blanches prises (par les noirs)
    # strip du bas = pièces noires prises (par les blancs)
    for captured_color, strip in (
        (WHITE, self._black_capture_strip), # blanches au-dessus
        (BLACK, self._white_capture_strip), # noires en dessous
    ):
        self._clear_box_children(strip)
        for piece in captured[captured_color]:
            strip.append(
                self._make_captured_piece_widget(piece, captured_color)
            )
```

5.3.3 Ordre d’affichage des captures

Les pièces ne sont pas affichées dans l’ordre de capture chronologique mais dans un **ordre visuel stable** : Tour > Cavalier > Alfil > Ferz > Pion. Cet ordre (défini par `CAPTURE_DISPLAY_ORDER`) est identique à celui utilisé dans les clients d’échecs modernes et rend le bandeau lisible d’un coup d’œil.

Pièce	Position	SVG source	Taille icône	Tooltip
Tour (Rook)	1 (gauche)	wR.svg / bR.svg	20×20 px	“Rook”
Cavalier (Knight)	2	wN.svg / bN.svg	20×20 px	“Knight”
Alfil	3	wA.svg / bA.svg	20×20 px	“Alfil”
Ferz	4	wF.svg / bF.svg	20×20 px	“Ferz”
Pion (Pawn)	5 (droite)	wP.svg / bP.svg	20×20 px	“Pawn”

5.4 Deux modes d’interaction : click-to-move et drag-and-drop

5.4.1 Click-to-move

Le flux est géré par `_on_click`. GTK 4 utilise `GestureClick` :

1. **Premier clic** : si la case contient une pièce du joueur courant, on calcule les coups légaux depuis cette case et on surligne les destinations via `queue_draw()`.
2. **Deuxième clic sur une destination** : le coup est joué via le callback `on_move_played`.
3. **Clic ailleurs** : la sélection est effacée.

5.4.2 Drag-and-drop avec GTK 4 `GestureDrag`

GTK 4 n’utilise plus le système DND de GTK 3 ; nous utilisons `GestureDrag` à la place :

```
def _on_drag_begin(self, gesture, x, y) -> None:
    """Sélectionne la pièce et calcule les coups légaux."""
    square = self._pixel_to_square(x, y)
    piece = self._board.get_piece_at(square)
    if piece is None or piece[1] != self._current_color:
        return
    self._drag_square = square
    self._dragging = True
    legal = self._engine.generate_legal_moves(self._board,
```



```

        self._current_color)
self._valid_moves = [m for m in legal if m.from_square == square]
self.queue_draw()

def _on_drag_update(self, gesture, dx, dy) -> None:
    """Met à jour la position de la pièce sous le curseur."""
    ok, start_x, start_y = gesture.get_start_point()
    if not ok:
        return
    self._drag_x = start_x + dx
    self._drag_y = start_y + dy
    self.queue_draw() # redessine à chaque mouvement

def _on_drag_end(self, gesture, dx, dy) -> None:
    """Joue le coup si la case de dépôt est valide."""
    ok, start_x, start_y = gesture.get_start_point()
    end_x, end_y = start_x + dx, start_y + dy
    target = self._pixel_to_square_from_pos(end_x, end_y)
    self._dragging = False
    self._drag_square = None
    for move in self._valid_moves:
        if move.to_square == target:
            self._valid_moves = []
            self.queue_draw()
            if self.on_move_played:
                self.on_move_played(move)
            return
    self._valid_moves = []
    self.queue_draw()

```

Pendant le drag, la pièce est rendue au-dessus de toutes les autres (dessinée en dernier dans `_draw_pieces`) à la position exacte du curseur (`self._drag_x`, `self._drag_y`).



FIGURE 19 – Drag-and-drop avec surbrillance des coups légaux

5.5 Thread IA et synchronisation GTK

Lancer l'IA dans le thread GTK principal bloquerait l'interface. Nous utilisons un thread Python dédié et `Glib.idle_add` pour repasser dans le thread GTK principal avant toute modification d'état :

```
def _auto_play_ai_turns(self) -> None:
    def ai_thread():
        while self._state is not None and \
            self._state.current_color in self._ai_players:
            if self._game_paused:
                time.sleep(0.05)
                continue
            ai = self._ai_players[self._state.current_color]
            move = ai.choose_move(self._state.board)
            if move is None:
                break
            done_event = threading.Event()
            # Glib.idle_add : retour dans le thread GTK
            Glib.idle_add(self._apply_ai_move, move, done_event)
            done_event.wait(timeout=5) # attend la fin de l'application

    thread = threading.Thread(target=ai_thread, daemon=True)
    thread.start()
```

`done_event` est un `threading.Event` qui synchronise le thread IA avec le thread GTK : l'IA ne calcule le coup suivant qu'une fois le coup précédent appliqué et affiché.

5.6 Horloge intégrée et CSS

L'horloge est implémentée avec `Glib.timeout_add(200, _on_timer_tick)` : un tick toutes les 200 ms rafraîchit les labels. L'apparence est entièrement définie en CSS (variable `CLOCK_CSS`) : chaque carte

horloge (clock-card-white, clock-card-black) a son propre dégradé. La classe clock-card-active ajoute un cadre doré (border: 2px solid #c7963e) sur la carte du joueur dont c'est le tour, et clock-card-cri passe le texte en rouge quand il reste moins de 20 secondes.

Le dialogue de configuration propose quatre modes de contrôle du temps : Bullet, Blitz, Rapid et Custom. Le mode Custom utilise deux Gtk . SpinButton (minutes et incrément) reliés à _on_custom_time_changed; le bouton "Start Match" ne devient actif que quand un preset ou une valeur custom est sélectionnée.



FIGURE 20 – Horloge en mode blitz avec cartes White/Black

5.7 Astuces et découvertes techniques

Réutilisation du CLI pour la sauvegarde/chargement. Plutôt que de réécrire la logique de sérialisation, le GUI instancie directement la classe CLI et réutilise ses méthodes _do_load et _save_to_file :

```
from shatranj.presentation.cli.cli import CLI
cli = CLI(verbose=False, debug=False)
cli._do_load([path]) # parsing complet avec commentaires
if cli._state is not None:
    self._state = cli._state # récupération du GameState parsé
```

Cela garantit que le format de fichier est identique entre CLI et GUI, et que les commentaires # et blocs { } sont correctement ignorés.

Conversion coordonnées pixel → case. Le plateau est redimensionnable. La conversion utilise _board_geometry :

$$\text{col} = \left\lfloor \frac{x - \text{offset}_x}{\text{sq}} \right\rfloor, \quad \text{row} = 7 - \left\lfloor \frac{y - \text{offset}_y}{\text{sq}} \right\rfloor$$

Le facteur 7− est indispensable : GTK mesure y depuis le haut de la fenêtre, alors que le Shatranj numérote les rangées depuis le bas.

Désactivation de l'interaction pendant le tour de l'IA. La méthode `set_interaction_enabled(False)` est appelée dès que l'IA commence à réfléchir : elle vide les états de sélection et de drag, et ignore tous les événements souris. Cela évite les conditions de course entre l'application d'un coup humain et le calcul de l'IA dans le thread secondaire.

6 Revue critique du projet

6.1 Ce qui a été réalisé

- Moteur de règles complet : validation, détection de l'échec, mat, pat et Bare King pour toutes les pièces.
- CLI interactif (prompt », Tab, historique, undo/redo, sauvegarde/chargement avec commentaires, hint, mode blitz avec chronomètre).
- GUI GTK 4 avec plateau SVG, deux thèmes de pièces, drag-and-drop, horloge CSS intégrée, dialogue de configuration complet, bandeaux de captures avec icônes SVG identiques à celles du plateau (20×20 px).
- IA fonctionnelle : AlphaBeta (profondeur 4), Iterative Deepening (avec limite de temps), et MCTS (500 simulations) avec TT partagée.
- Internationalisation français/anglais via `gettext`.
- Tests automatisés (`pytest`) avec couverture (`pytest-cov`).

6.2 Ce qui n'a pas été réalisé

- **Iterative Deepening dans le GUI** : l'algorithme `IterativeDeepening` est disponible dans le CLI mais n'est pas exposé dans le dialogue de configuration du GUI ; seuls AlphaBeta, Minimax et MCTS y sont proposés.
- **Jeu en réseau** : module `domain/network/` prévu (architecture client-serveur TCP avec découverte UDP) mais non implémenté dans le GUI.
- **Machine Learning** : intégration Keras pour remplacer UCT dans le MCTS non développée faute de temps.

6.3 Performances

L'AlphaBeta à profondeur 4 répond en moins de cinq secondes sur des positions ouvertes. L'Iterative Deepening avec une limite de 5 secondes est plus robuste en fin de partie : il explore plus profondément quand le facteur de branchement diminue. La table de transposition partagée réduit le nombre de nœuds évalués de 30 à 50 % sur les positions répétées. Le MCTS avec 500 simulations offre une variété de jeu intéressante mais est plus lent : environ 2 à 3 secondes par coup en position ouverte.

6.4 Cas limites et bugs identifiés

- **Alfil en finale** : ses 8 cases fixes de même couleur le rendent quasi inutilisable. L'évaluateur devrait pondérer l'Alfil différemment selon la phase de jeu (ouverture, milieu, finale).
- **Détection de répétition triplée** : comparaison en $O(n)$ sur l'historique complet ; peut ralentir les parties très longues.
- **Thread IA et undo** : si le joueur clique sur Undo pendant que l'IA calcule, le `done_event.wait(timeout=5)` peut expirer. Un verrou supplémentaire serait nécessaire pour une synchronisation parfaite.

6.5 Améliorations possibles

- Développer le module réseau (TCP/UDP, tableau des scores).
- Ajouter l'ordonnancement des coups dans AlphaBeta [?] (captures en premier) pour maximiser les coupures.
- Pondérer l'Alfil selon la phase de jeu dans l'évaluateur.
- Utiliser un verrou explicite pour la synchronisation thread IA / undo.
- Intégrer Keras pour remplacer UCT dans le MCTS.

A Bilan des fonctionnalités (F1–F40)

TABLE 1 – Bilan des fonctionnalités (F1–F40)

ID	Description	État
Lancement et configuration		
F1	Options <code>-h</code> , <code>-V</code> , <code>-v</code> , <code>-d</code>	Fait
F2	Fichier de configuration <code>.shatranjrc</code>	Fait
F3	Internationalisation fr/en via <code>gettext</code>	Fait
Options au lancement		
F4	Interface CLI par défaut	Fait
F5	Mode Blitz avec chronomètre	Fait
F6	Mode Contest (entrée fichier, sortie <code>stdout</code>)	Fait
F7	Interface graphique GTK (<code>-g</code>)	Fait
F8	Mode IA joueur (<code>-a W/B/A</code>)	Fait
Gestion du jeu		
F9	Validation des coups et mise à jour de l'état	Fait
F10	Gestion des erreurs utilisateur (messages <code>stderr</code>)	Fait
F11	Undo / Redo	Fait
F12	Hints (suggestion de coup via IA)	Fait
F13	Blitz : décompte, pause, timeout	Fait
Interface CLI		
F14	Shell interactif avec prompt »	Fait
F15	Proposition de sauvegarde avant de quitter	Fait
F16	Édition de la ligne de commande (<code>readline</code>)	Fait
F17	Historique des commandes (flèches, <code>Ctrl+R</code>)	Fait
F18	Complétion Tab	Fait
F19	Notation algébrique des coups	Fait
Format de sauvegarde		
F20	Sauvegarde / restauration d'une partie	Fait
F21	Commentaires dans la sauvegarde (<code>#</code> , <code>{ }</code>)	Fait
F23	Format du plateau dans <code>[game]</code>	Fait
F24	Format de l'historique dans <code>[history]</code>	Fait
Bitboards		
F25	Module bitboard (primitives bit-à-bit)	Fait
F26	Génération et application des coups	Fait
Interface graphique		
F27	Menus File / Game	Fait
F28	Raccourcis clavier	Fait
F29	Drag-and-Drop	Fait
Intelligence artificielle		
F30	Temps de réflexion borné	Fait
F31	Minimax avec élagage α - β	Fait
F32	Fonctions d'évaluation (material, positional, advanced)	Fait
F33	Approfondissement itératif	Fait
F34	Profondeur de recherche configurable	Fait

Suite page suivante

ID	Description	État
F35	Monte Carlo Tree Search (MCTS)	Fait
F36	Sélection par Machine Learning (Keras)	Non fait
F37	Tables de transposition (Zobrist)	Fait

Jeu en réseau

F38	Découverte de serveurs (UDP) et connexion TCP	Fait
F39	Serveur multi-clients, tableau des scores	Fait
F40	Système d'invitations, statuts des joueurs	Fait

B Valeurs des pièces

Pièce	Valeur	Commentaire
Pion (Pawn)	1	unité de base
Alfil	2	8 cases fixes de même couleur
Ferz	2	1 case en diagonale seulement
Cavalier (Knight)	6	saut en L, très mobile
Tour (Rook)	9	ligne droite illimitée
Shah (Roi)	0	non capturé, à protéger